

Verification — 검증

🔥 SystemVerilog/UVM·assertion·coverage·constrained random으로 RTL의 기능 정확성을 수렴 검증하고, DFT(MBIST/scan/ATPG)로 제조 결함을 테스트 · 🏷️ P1 · 🧠 메모리 브릿지: 메모리 컨트롤러/PHY는 UVM 환경으로 기능 검증, DRAM array는 MBIST-BISR로 제조 후 결함 test-repair. DV(기능)와 DFT(제조 test)는 다른 축.

핵심 개념

- **SystemVerilog**: HDL+HVL. class(OOP)·constrained random·assertion·coverage·interface·mailbox 등 검증 기능 내장.
- **UVM 계층**: test → env → **agent**(driver+monitor+sequencer) → scoreboard. sequence/sequence_item, phase(build/connect/run), factory·config_db·TLM.
- **driver**: sequence item을 DUT 핀으로 구동(interface 통해).
- **monitor**: 핀을 수동 관찰해 transaction 복원(체크/coverage용).
- **sequencer**: sequence를 arbitration해 driver로 item 전달.
- **scoreboard**: DUT 출력 vs reference model 비교.
- **agent**: driver+monitor+sequencer 묶음(active=구동, passive=관찰만).
- **SVA**: SystemVerilog Assertion. immediate vs **concurrent**(clocked, temporal). property/sequence, |→ (같은 cycle) · |⇒ (다음 cycle) implication. 시간적 동작 검증.
- **Coverage**: **code**(line/branch/toggle/FSM/condition)=코드가 실행됐나, **functional**(covergroup/coverpoint/cross)=시나리오가 일어났나. code 100% ≠ functional 완료.
- **Constrained random**: 제약 안에서 stimulus 랜덤 생성 → corner case 자동 hit, directed보다 효율적.
- **검증 수렴(coverage signoff)**: regression + coverage를 목표치까지 채우고 미달 hole을 directed/제약 조정으로 닫음.
- **DV vs DFT**: DV=기능 검증(설계가 맞나). DFT=제조 결함 test 용이성(scan/ATPG/BIST).
- **DFT**: **scan**(FF를 shift chain으로 묶어 제어/관찰성↑), **ATPG**(stuck-at/transition fault 패턴 자동 생성), **MBIST**(메모리 자체 test, march algorithm), **redundancy/repair**(spare row/col + BISR로 결함 cell 대체 →yield).

예상 면접 Q&A

Q1. UVM testbench의 주요 컴포넌트와 역할을 설명하세요.

A. 최상위 test가 env를 생성하고, env 안에 agent와 scoreboard가 있습니다. agent는 sequencer·driver·monitor를 묶은 단위로, sequencer가 sequence(자극 시나리오)를 arbitration해 transaction item을 driver에 주면 driver가 그것을 interface 통해 DUT 핀으로 구동합니다. monitor는 핀을 수동으로 관찰해 transaction을 복원하고, scoreboard가 그 관찰값을 reference model 기대값과 비교해 pass/fail을 판정합니다. coverage collector가 monitor 출력으로 functional coverage를 모읍니다. 재사용은 factory·config_db로 지원합니다. - L, 꼬리질문: active와 passive agent 차이? → active는 sequencer+driver로 구동까지, passive는

monitor만 뒤 관찰/coverage 전용(이미 다른 곳이 구동할 때). - 🧠 메모리 브릿지: 메모리 컨트롤러 검증에서 host측 active agent로 트랜잭션 구동, DRAM측 passive로 프로토콜 관찰.

Q2. SVA(assertion)는 무엇이고 immediate와 concurrent의 차이는?

A. SVA는 설계가 지켜야 할 시간적/논리적 속성을 선언적으로 기술해 시뮬·formal에서 자동 검증하는 수단입니다. immediate assertion은 절차 코드 안에서 그 순간의 조건을 즉시 체크(조합적)하고, concurrent assertion은 clock에 동기화돼 여러 cycle에 걸친 sequence/property(예: req 후 2 cycle 내 ack)를 검증합니다. $a \rightarrow b$ 는 같은 cycle, $a \Rightarrow b$ 는 다음 cycle 함의입니다. 버그를 발생 지점에서 바로 잡고 coverage(cover property)로도 씁니다. - L, 꼬리질문: assertion이 검증에 좋은 이유? → 버그가 출력까지 전파되길 기다리지 않고 내부에서 즉시 포착 → debug 시간 단축. formal로 exhaustive 증명도 가능. - 🧠 메모리 브릿지: CDC 동기화 프로토콜·메모리 command 타이밍 규칙을 SVA로 명세·검증(내 CDC 업무와 직결).

Q3. code coverage와 functional coverage의 차이, 그리고 둘 다 필요한 이유는?

A. code coverage는 RTL의 line/branch/condition/toggle/FSM이 시뮬에서 실제로 실행됐는지를 보는 구조적 지표로, 도구가 자동 추출합니다. functional coverage는 설계 의도(어떤 시나리오·corner·교차 조건이 일어났는가)를 설계자가 covergroup으로 정의해 측정합니다. code coverage 100%여도 "특정 조합 상황"을 안 만들었으면 기능 hole이 남으므로, code(코드를 다 건드렸나)와 functional(의도한 상황을 다 봤나)을 함께 봐야 검증이 완결됩니다. - L, 꼬리질문: coverage 100%면 버그 없음 보장? → 아니다. coverage는 "안 본 곳"을 알려줄 뿐, 본 곳이 맞는지는 checker/scoreboard·assertion이 보장. - 🧠 메모리 브릿지: 메모리 프로토콜의 모든 command 조합·timing corner를 cross coverage로 정의해 hole 추적.

Q4. constrained random verification이 directed test보다 좋은 점은?

A. directed test는 사람이 시나리오를 하나씩 작성해 예상한 경우만 보지만, constrained random은 제약(유효 범위·프로토콜) 안에서 자극을 무작위 생성해 사람이 미처 생각 못 한 corner case까지 자동으로 도달합니다. 같은 노력으로 더 넓은 상태공간을 커버하고, coverage 피드백으로 부족한 부분에 제약을 조정해(coverage-driven) 효율적으로 수렴합니다. 단, 무엇이 옳은지는 scoreboard/reference가 판단해야 합니다. - L, 꼬리질문: 그래도 directed가 필요한 경우? → 도달이 매우 어려운 특정 corner나 회귀 버그 재현은 directed로 못 박음. - 🧠 메모리 브릿지: refresh 충돌·bank 경합 같은 회귀 timing corner를 random으로 자동 발굴.

Q5. 검증 수렴(coverage sign-off)은 어떻게 판단하나요?

A. 단순히 testcase가 통과하는 게 아니라, (1)code+functional coverage가 목표치(보통 100% 또는 합의된 수준)에 도달, (2)모든 assertion pass, (3)회귀(regression)가 안정적으로 green, (4)남은 coverage hole이 분석돼 unreachable로 waiver되거나 directed로 채워졌는지를 봅니다. 즉 "안 본 곳이 없고, 본 곳은 다 맞다"가 성립할 때 sign-off합니다. - L, 꼬리질문: unreachable coverage는? → formal/분석으로 도달 불가를 입증해 exclusion(waiver). CDC waiver와 같은 관리 철학. - 🧠 메모리 브릿지: 대규모 메모리 IP는 coverage 자동 분석·회귀 자동화가 품질의 핵심(내 EDA 자동화 경험과 연결).

Q6. DV와 DFT는 어떻게 다른가요?

A. DV(Design Verification)는 설계가 사양대로 올바르게 동작하는가를 시뮬·formal로 검증하는 기능 검증입니다. DFT(Design for Test)는 제조된 chip에 물리적 결함이 있는지 테스트할 수 있게 회로를 설계하는 것으로, scan chain·ATPG·BIST를 넣어 controllability/observability를 높입니다. DV는 설계 단계 "맞게

만들었나", DFT는 제조 후 "제대로 만들어졌나"를 봅니다. - L, 꼬리질문: fault model 예? → stuck-at(0/1 고착), transition(천이 지연), bridging. ATPG가 이 모델로 패턴 생성. - 🧠 메모리 브릿지: 메모리는 array 결함률이 높아 DFT 비중이 큼 → MBIST+BISR가 yield를 좌우.

Q7. MBIST, scan/ATPG, redundancy/repair(BISR)를 설명하세요.

A. scan은 일반 FF를 scan FF로 바꿔 shift chain으로 연결, 임의 상태를 넣고 꺼내 controllability/observability를 확보하고, ATPG가 stuck-at/transition fault를 잡는 테스트 패턴을 자동 생성합니다. 메모리는 핀으로 일일이 못 보므로 **MBIST**가 on-chip에서 march 알고리즘(0/1 write-read 패턴)으로 cell-sense-decoder 결함을 자가 진단합니다. 발견된 결함은 **redundancy**(여분의 spare row/column)와 **BISR**(Built-In Self Repair)로 결함 주소를 spare로 재매핑해 살려, 일부 불량 cell이 있어도 chip을 살려 yield를 올립니다. - L, 꼬리질문: repair는 언제 하나? → 제조 test 시점에 결함 주소를 fuse(e-fuse)나 비휘발 저장에 기록해 영구 재매핑(hard repair) 또는 부팅 시 soft repair. - 🧠 메모리 브릿지: DRAM/HBM은 spare row/col-BISR이 표준 — 신입이 "메모리는 repair로 yield를 산다"를 알면 갭을 메우는 강점.

30초 암기 요약

- SystemVerilog=HDL+HVL(class/random/assertion/coverage).
UVM=test→env→agent(driver+monitor+sequencer)→scoreboard.
- driver=핀 구동, monitor=수동 관찰·복원, sequencer=sequence arbitration, scoreboard=기대값 비교.
agent active/passive.
- SVA: immediate(즉시) vs concurrent(clocked·temporal). |→ 같은 cycle, |=> 다음 cycle. 버그를 내부에서 즉시 포착.
- coverage: code(실행됐나)+functional(시나리오 봤나). 둘 다 필요. coverage≠정확성(checker가 보장).
- constrained random=제약 내 랜덤으로 corner 자동 도달. directed는 회귀/회귀 못 박기용.
- 수렴 sign-off=coverage 목표+assertion pass+regression green+hole 분석/waiver.
- DV=기능 검증(맞게 만들었나), DFT=제조 test 용이성(제대로 만들어졌나).
- DFT: scan+ATPG(stuck-at/transition), MBIST(march, 메모리 자가 test), redundancy+BISR(spare로 재매핑→yield).